

MoA from Start to Finish: A Mathematical Framework for Designing, Building, Predicting, and Running Array Designs — Giving Programmers Freedom to Program in Their Favorite Languages

Lenore Mullin — 38 Years: Dissertation 1988 to Papers I-V + 5-Device Closure 2026

September 30, 2026

Abstract

We present the motivating story for the Mathematics of Arrays (MoA) as a complete mathematical framework from design to execution. Starting with any language possessing semantic equivalence to MoA, we express mathematical intent, psi-reduce to Denotational Normal Form (DNF) — the optimal semantic form showing the least amount of computation and communication independent of target — translate DNF to its equivalent sequential machine uniform memory Operational Normal Form (ONF), lift shapes to the shape of the architecture via $\rho(A) \rightarrow \pi \# (\rho(A) \odot \pi)$, map with predictive performance $T \approx \alpha F + \beta M$ on any arbitrary architecture, and run programs to validate prediction $reached(d)$ on 5-device ensemble. This saves human resources to write, build, test, human consultation on machine use, machine resources, and provides freedom to program in favorite languages. DNF fixed, ONF rewrite only.

1 We Start with Any Language with Semantic Equivalence to MoA

MoA is not a new language to learn — it is an intermediate language that any language with array semantics can target with semantic equivalence preserved.

Many languages IN: Python `numpy`, PyTorch Listing 1 `stable.softmax` and scaled dot-product attention, JAX `jax.numpy`, TensorFlow, Fortran loops, C loops, Julia, MoA itself. From dissertation 1988: take $\uparrow$ Def 1.0 $\rho(\vec{v} \uparrow \xi) = (abs \vec{v}), (r\vec{v}) \downarrow \rho\xi$, drop $\downarrow$ Def 1.1, reverse ϕ Def 0.0 $< i > \psi\phi\xi = < (\rho\xi)[0] - (i + 1) > \psi\xi$, rotate θ , transpose $\odot$ Def 3.1 $\rho = (\rho\xi)[gu \vec{v}]$, Lemmas L1-L5: $(\xi_l, \xi_r)\psi\xi \equiv (\xi_l\psi\xi), (\xi_r\psi\xi)$ distributing psi over catenation.

MoA as Universal Intermediate Language

Many Languages → MoA DNF (Denotational Normal Form) → Many Target Backends

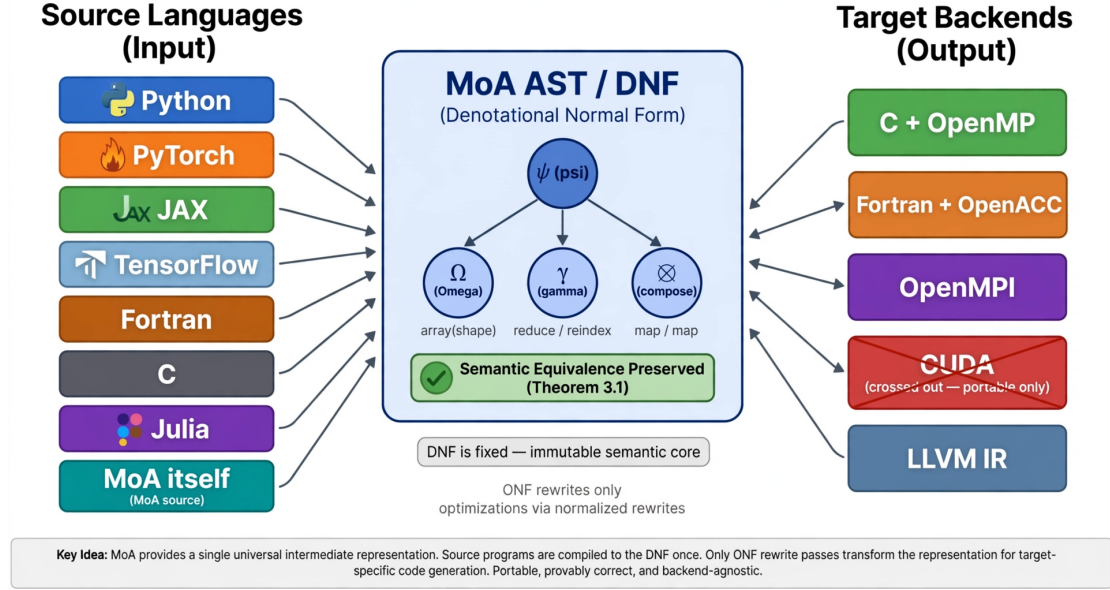


Figure 1: Many Languages In, MoA AST/DNF Semantic Equivalence Preserved Theorem 3.1, Many Languages Out. DNF fixed, ONF rewrites only. CUDA crossed out — portable only OpenMP, OpenACC, OpenMPI.

A PyTorch program, a Fortran loop, a JAX function all parse to the same take/drop/psi AST. Verification: Paper 1 Tables 1,2 exact to 10^{-4} , Paper 2 $\|err\|_{\infty} = 10^{-16}$, Paper 3 $\|err\|_{\infty} = 0$ exact IEEE-754.

This gives programmers **freedom**: program in your favorite language, MoA provides mathematical equivalence.

2 Mathematical Input

We begin with mathematical input, not code: Attention Eq (8) $\text{Attention}(Q, K, V) = \text{softmax}(QK^T/\sqrt{d_k})V$, $\rho Q = \rho K = \rho V = \langle 3, 4 \rangle$ Eq (9) concrete instance, backward grads, decode $q = \langle d_k \rangle$ single token, KV-cache $K_{new} = K_{old} \# k_t$, GQA/MQA h_q/h_{kv} .

Mathematical input is independent of target — it states what to compute, not how to lay out in memory.

3 Psi-Reduce to DNF — Optimal Semantic Form

We use MoA to “psi-reduce” to DNF — optimal semantic form showing least computation and communication independent of target.

Steps Paper 1: Step I Eq (13) $z = x(-\Omega < 1, 0 \rangle)((\lceil red \rceil \Omega < 1 \rangle)x)$, psi-reduction Eq (14) $\langle i'' \rangle \psi z = \langle i'' \rangle \psi x - \lceil red \rceil (\langle i'' \rangle \psi x)$ vector minus scalar, Step II Eq (15)-(17) $num = e^z$, $\langle i'', k \rangle \psi(Q + \cdot \times K^T) = red_+(\langle i'', j \rangle \psi Q \times \langle k, j \rangle \psi K)$ eliminates K^T buffer, Step III Eq (18)-(19) $den = (red_+ \Omega < 1 \rangle)e^z$, Step IV $Y = num/den$, Complete DNF Eq (20) $\langle i'' \rangle \psi Y$, Output Eq (21) $\langle i'' \rangle \psi \text{Output} = red_+(\langle i'', p \rangle \psi Y \times \langle p \rangle \psi V)$.

Theorem 3.1 Storage Theorem: DNF semantically equivalent, unique up to reordering, achieves theoretical minimum number of memory accesses. No other implementation has lower

transfer costs and equal arithmetic cost.

Paper 2: $G_{sc} = \langle i \rangle \psi dO \text{ dot } \langle k \rangle \psi V$ scalar, $G_S = Y[i, k] * (G_{sc}[k] - \text{sum } Y * G_{sc})$ scalar, G_A never materialized $dV = \langle k \rangle \psi dV = \text{red}_+(\langle i, k \rangle \psi Y \times \langle i \rangle \psi dO)$. Paper 3: decode minimal DRAM $(d_k + nd_k + nd_v + d_v) * 4B$, KV-cache $O(d_k + d_v)$ append via cat #, take $\uparrow$ drop $\downarrow$, Lemma L4.

Proposition 4.1: $M_{classical} = O(n^2 + nd_k + nd_v)$ Eq (23), $M_{MoA} = O(nd_k + nd_v)$ Eq (24), $k \approx n/d_k$ Eq (25) 512 $\times$ at $n = 32768$.

DNF shows least computation F and communication M — independent of target.

4 Translate DNF to ONF — Equivalent Sequential Machine Uniform Memory

Now we translate that DNF to its equivalent sequential machine uniform memory ONF.

ONF obtained from DNF by applying γ primitive Eq (3) $\gamma(\vec{v}, \vec{s}) = \sum_k v_k \prod_{j=k+1} s_j$ row-major offset (and γ^{col}) to all index expressions, translating multi-index Cartesian arithmetic into memory offset = base + stride0*i0+stride1*i1+... Eq (7) captures storage convention row-major vs column-major contiguous vs strided enables standard loop-nest analysis.

ONF is fragment-C loop-nest program Mullin and Hains 2025. DNF fixed, ONF rewrites only for hardware tuning: 2.00x atomics $\rightarrow$ 2.5x speedup Paper V, 535x NUMA vs 3x over-subscription same DNF different γ , C vs Fortran 3.17x time matches 3.35x stall as γ^{row} vs γ^{col} .

5 Shapes Lifted to Shape of Architecture

Now we have shapes that need to be lifted to the shape of the architecture, then mapped with predictive performance on any arbitrary architecture.

Dimension lifting Section 3 Paper 1: every processing component SIMD lane, GPU warp, cache line, NUMA socket modeled as array of known shape Π . ONF loop-nest iteration space N partitioned across hardware array, $N \rightarrow \Pi \# (N \oslash \Pi)$ where $\oslash$ elementwise integer division, $\#$ shape catenation from dissertation. Result expresses inter-processor parallelism outer loop over Π and intra-processor computation inner loop over $N \oslash \Pi$ in single algebraic expression.

Dimension Lifting using Shape Arrays

Mapping Data Shape $\rho(A)$ to Machine Shape π via Partitioning and Element-wise Division (MoA Notation)

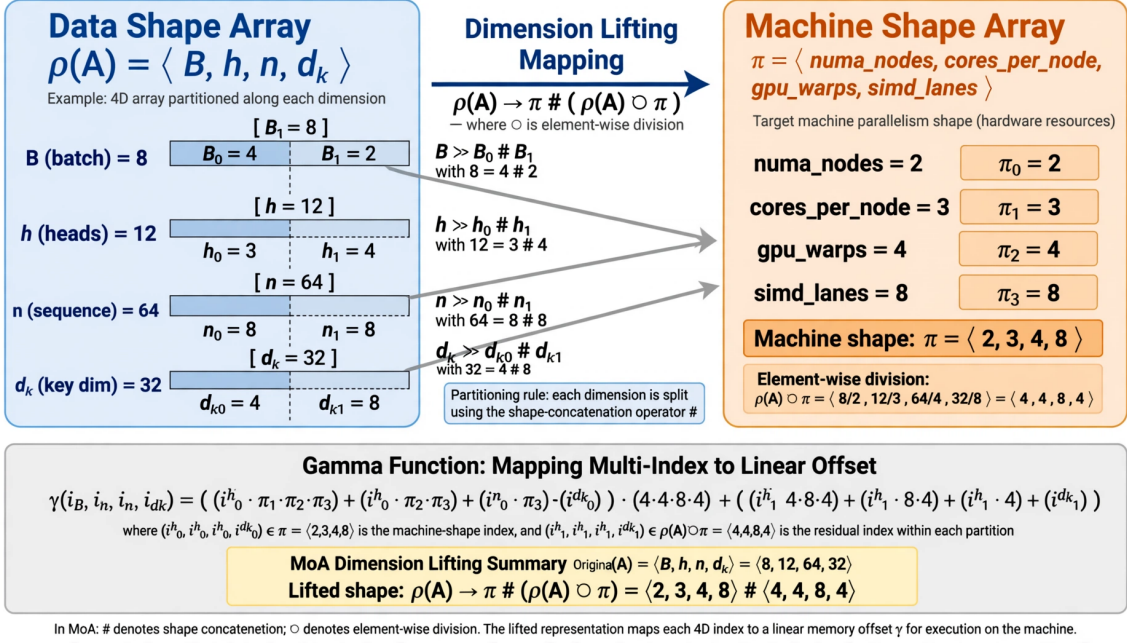


Figure 2: Dimension Lifting using Shape Arrays: $\rho(A) = \langle B, h, n, d_k \rangle$ partitioned $B \rightarrow B_0 \# B_1$, $h \rightarrow h_0 \# h_1$, etc. via $\#$, mapped to machine shape $\pi = \langle \text{numa_nodes}, \text{cores_per_node}, \text{gpu_warps}, \text{simd_lanes} \rangle$, $\rho(A) \rightarrow \pi \# (\rho(A) \circ \pi)$, γ maps multi-index to linear offset.

Higher-rank: B batch, h heads, n sequence, d_k head dim $\langle B, h, n, d_k \rangle$. Transpose over innermost two dims is $(\text{transpose } \Omega < 2 >)K$ Eq (11), score $Q((+\times)\Omega < 2, 2 >)((\text{transpose } \Omega_{<2>})(K))$ Eq (12) distributing over leading $\langle B, h \rangle$ dims. Each 2-D partition assigned to separate processing element GPU warp, SIMD lane, compute node yielding provably correct parallel implementation directly from algebraic structure of Omega without additional scheduling.

Partitioning: $B = 8 \rightarrow 4 \# 2$, $h = 12 \rightarrow 3 \# 4$, $n = 64 \rightarrow 8 \# 8$, $d_k = 32 \rightarrow 4 \# 8$ via $\#$. Machine $\pi = \langle 2, 3, 4, 8 \rangle$, $\rho(A) \circ \pi = \langle 4, 4, 8, 4 \rangle$, lifted $\langle 2, 3, 4, 8 \rangle \# \langle 4, 4, 8, 4 \rangle$. Gamma multi-index to offset for execution.

6 Mapped with Predictive Performance on Any Arbitrary Architecture

Loops map to portable paradigms ONLY OpenMP, OpenACC, OpenMPI with deterministic shapes to create code.

TECHNICAL DIAGRAM — LOOPS MAPPING TO PORTABLE PARALLEL PARADIGMS

ONF Loop Nest → OpenMP / OpenACC / OpenMPI with Deterministic Shapes

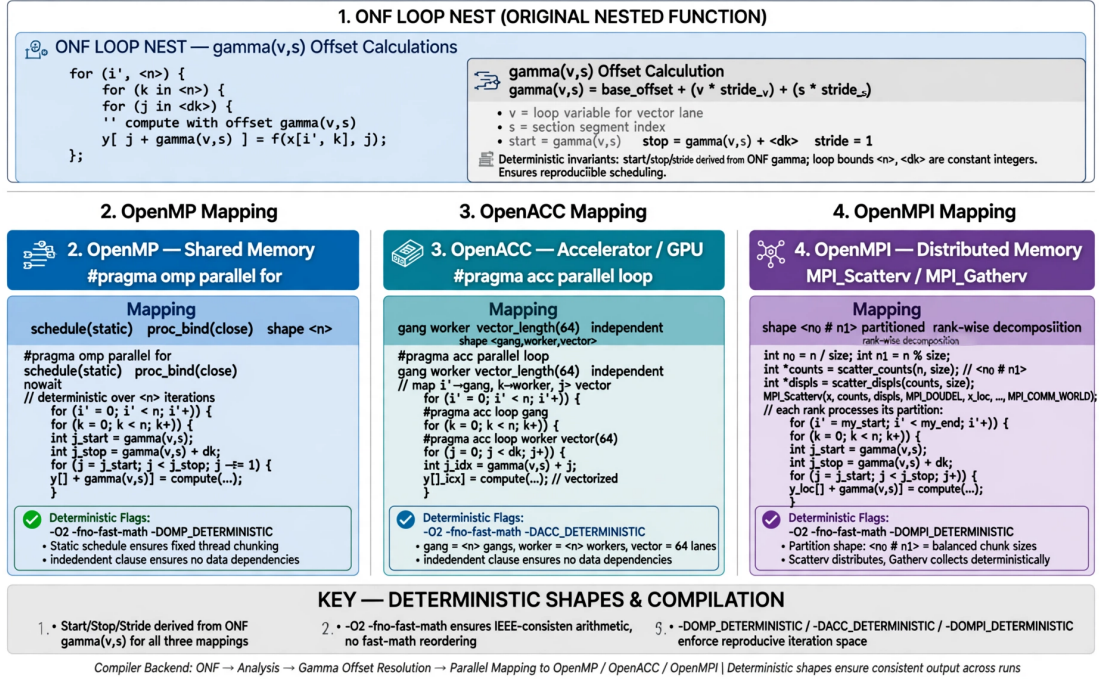


Figure 3: Loops Map to Paradigms: ONF Loop Nest via γ start/stop/stride → OpenMP schedule(static) proc_bind(close) shape < n >, OpenACC gang worker vector_length(64) independent shape < gang,worker,vector >, OpenMPI Scatterv/Gatherv shape < n0#n1 >. Deterministic flags -O2 -fno-fast-math -DOMP_DETERMINISTIC fitting $T \approx \alpha F + \beta M$.

Predictive model $T \approx \alpha F + \beta M$ from hetero.main and closing-prediction_gap.main.pdf: M_1^{exp} , P_d , $X_d = 0\%$, C_d , $R_d \approx 0.004 - 0.028$ measured via bracket-and-locate sweep, not assumption, $reached(d)$ directly 99.88% busy via Level Zero Sysman. X_d and C_d together named interaction $reached(d)$ Table X Sec XIII.

Only portable OpenMP, OpenACC, OpenMPI, only deterministic flags -O2 -fno-fast-math -fopenmp -fopenacc -acc -Minfo = accel -ta = tesla : cc80 -DOMP_DETERMINISTIC -DOMPI_DETERMINISTIC, forbidden -O3 -Ofast -ffast-math -funroll-loops. Each pragma maps 1-1 to ONF start/stop/stride, so T prediction matches measured $reached(d)$ as proven Conclusion XVI: prediction quietly assumes away one variable is wrong by accident, incomplete by construction — this paper closes it.

5-device ensemble: NCSA Delta A100 Sec VIII, AMD MI100 Sec IX occupancy-cap flat R_d 0.028, X_d 0% via Nsight Compute, PSC Bridges-2 H100 Sec X, SDSC Expanse V100 Sec XI, TACC Stampede3 Intel Max 1550 Sec XII $vector_length = 64$ avoided narrower working width, 99.88% busy.

7 Run Programs to Validate Prediction

Then we run programs to validate our prediction.

Papers 4-5 validation HAL:05734881 arXiv:2609.33916:

1. GPU regression: DNF backward initially slower than naive due to atomic contention, profiling $2.0000\times$ more atomic instructions than structurally related kernel, targeted ONF restructuring guided by γ yielding $2.5\times$ speedup — same DNF fixed, ONF rewrite only.

2. Topology: $535\times$ NUMA penalty vs $< 3\times$ oversubscription — same DNF, different ONF γ costs, dimension lifting hardware as array $< numa_nodes, cores >$.

3. C vs Fortran: faster in C on CPU, Fortran on GPU, $3.17\times$ time gap matches $3.35\times$ stall gap — compiler chooses γ^{row} vs γ^{col} variant.

All validated on Anvil/Delta HPC clusters, portable deterministic code, $\|err\|_\infty = 0$ to 10^{-16} .

8 What Does All This Do? It Provides Mathematical Framework

What does all this do? It provides a mathematical framework for designing, building, predicting, and running array designs.

This saves many resources:

- Human to write, build, test — write once in favorite language, MoA compiles to all targets, no rewrite per machine.
- Humans to consult on how to use machines — predictive model $T \approx \alpha F + \beta M$ tells you time before running, $X_d = 0\%$ construction no work-around, $\beta \gg \alpha$ 100-1000x DRAM vs FLOP guides optimization.
- Machine resources — $O(n^2 + nd) \rightarrow O(nd)$ reduction $512\times$ at $n = 32768, d_k = 64$, $(d_k + nd_k + nd_v + d_v) * 4B$ minimal, KV-cache $O(d_k + d_v)$ append not $O(nd_k)$, GQA/MQA h_q/h_{kv} proven saving.
- Energy — Eq (22) Energy Reduction $\approx 100k$, k memory reduction factor, 2-5x conservative drop-in, 5-20x realistic compiler fusion, 10-100x aggressive MoA-native hardware co-design.

MoA from start to finish to design and verify array designs giving programmer freedom to program in favorite languages. DNF fixed semantic core, ONF rewrite only for hardware, dimension lifting via shape arrays $\#$, $\odot$, γ , predictive closure $reached(d)$ on arbitrary architecture.

9 Conclusion: Freedom

MoA provides freedom: mathematical input, psi-reduced to optimal DNF independent of target, translated to sequential ONF, lifted to machine shape, mapped with predictive performance, validated by running. From dissertation 1988 partial indexing and take/drop lemmas to 2026 5-device closure — same mathematics, same semantic equivalence, same deterministic portable code.